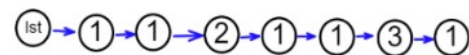


```

1 public class Person {
2     private String name;
3     public String getName() { return this.name; }
4 }
5
6 public class Room {
7     private Node<Person> people;
8     public Node<Person> getPeople() { return this.people; }
9
10    public Room() {
11        this.people = null;
12    }
13 }
14
15 public static boolean contains(Node<Room> hotel, int name)
16 {
17     while (hotel != null) {
18         Node<Person> lst = hotel.getValue().getPeople();
19         while (lst != null) {
20             if (lst.getValue().equals(name)) {
21                 return true;
22             }
23             lst = lst.getNext();
24         }
25         hotel = hotel.getNext();
26     }
27     return false;
28 }

```



removeAll(lst, 1)

```

1 public static Node<Integer> removeAll(Node<Integer> lst, int
2     // מוחקים את הצמתים בראש הרשימה
3     while (lst != null && lst.getValue() == value) {
4         lst = lst.getNext();
5     }
6     // מה אם מחקנו הכול?
7     if (lst == null) {
8         return null;
9     }
10    // מוחקים מאמצע של רשימה
11    Node<Integer> prev = lst;
12    while (prev.hasNext()) {
13        if (prev.getNext().getValue() == value) {
14            prev.setNext(prev.getNext().getNext());
15        } else {
16            prev = prev.getNext();
17        }
18    }
19    return lst;
20 }

```

```

1 class Stack {
2     private Node<Integer> lst;
3
4     public Stack() {
5         this.lst = null;
6     }
7
8     public void push(int x) {
9         this.lst = new Node(x, this.lst);
10    }
11
12    public int pop() {
13        int x = lst.getValue();
14        this.lst = lst.getNext();
15        return x;
16    }
17 }
18
19 Stack stack = new Stack();
20 stack.push(123);
21 stack.push(456);
22 print(stack.pop()); // 456
23 print(stack.pop()); // 123

```

```

1 class Store {
2     private Node<Game> lst;
3
4     public int remove(int n, int pr) {
5         int count = 0;
6         // מוחקים את הצמתים בראש הרשימה...
7         while (this.lst != null && count < n &&
8             this.lst.getValue().getPrice() == pr)
9         {
10            this.lst = this.lst.getNext();
11            count++;
12        }
13        ~r hr return-what
14        // מה אם מחקנו הכול?
15        if (this.lst == null) {
16            return count;
17        }
18        // מוחקים מאמצע של רשימה...
19        Node<Game> prev = this.lst;
20        while (prev.hasNext() && count < n) {
21            if (prev.getNext().getValue().getPrice() == pr)
22                prev.setNext(prev.getNext().getNext());
23            count++;
24        } else {
25            prev = prev.getNext();
26        }
27    }
28    return count;
29 }
30 }

```

```

1 class Store {
2     private Node<Game> lst;
3
4     public int minPrice() {
5         int min = Integer.MAX_VALUE;
6         Node node = this.lst;
7         while (node != null) {
8             min = Math.min(min, node.getValue().getPrice());
9             node = node.getNext();
10        }
11        return min;
12    }
13
14    public int removeCheap(int num) {
15        int sum = 0;
16        while (num > 0) {
17            int min = minPrice();
18            int count = remove(num, min);
19            sum += min * count;
20            num -= count;
21        }
22        return sum;
23    }
24 }

```

```

// מחזירה אמת אם איקס נימצא בתור
// כול המספרים בתור חיוביים
1 public static boolean contains(Queue<Integer> q, int x) {
2     boolean found = false;
3     q.insert(0); // מופתח שאין אפס בתור
4     while (q.head() > 0) { // בודקים מה הוא האביר הבא
5         if (q.head() == x) { // בלי להוציא אותו מהתור
6             found = true;
7         }
8         q.insert(q.remove()); // גלגל אותו!
9     }
10    q.remove(); // להוציא אפס שהיכנסנו
11    return found;
12 }

```

mmm(q, z=4): false

התור q	num	y	y < z	q.head() == num
[1,3,1,1,1,2]	1	0	true	true
[1,3,1,1,1,2,0]		1		
[3,1,1,1,2,0,1]	3	1	true	false
[1,1,1,2,0,1,3]		1		
[1,1,2,0,1,3,1]	1	2	true	true
[1,2,0,1,3,1,1]		3		
[2,0,1,3,1,1,1]	2	1	true	false
[0,1,3,1,1,1,2]		1		
[1,3,1,1,1,2]				

```

1 public static boolean mmm(Queue<Integer> q, int z) {
2     q.insert(0);
3     int num = q.head();
4     int y = 0;
5     while (q.head() > 0) {
6         if (y < z) {
7             if (q.head() == num) {
8                 y++;
9             } else {
10                num = q.head();
11                y = 1;
12            }
13        }
14        q.insert(q.remove());
15    }
16    q.remove();
17    return y == z;
18 }

```

הפעולה בודקת האם יש בתור סידרה בעורך z של מספרים זהים סיבוכיות זמן הריצה: $O(n)$

what(q, n=5): 3

התור q	הערך n	mmm(q, n) == true	ערך מוחזר
[1,3,1,1,1,2]	6	false	3
[1,3,1,1,1,2]	5	false	3
[1,3,1,1,1,2]	4	false	3
[1,3,1,1,1,2]	3	true	3

הפעולה מחפשת:

את האורך המקסימלי לא גדול מ-n של רצף של מספרים זהים בתור

סיבוכיות זמן ריצה: $O(n^2)$
עבור כל ערך של n עוברים על התור

```

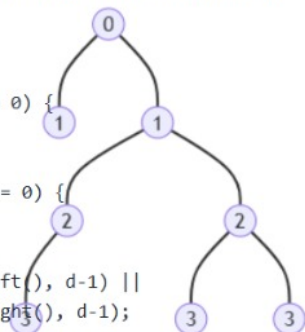
1 public static int what(Queue<Integer> q, int n) {
2     if (mmm(q, n)) {
3         return n;
4     }
5     return what(q, n - 1);
6 }

```

```

1 public static boolean isDeeper(BinNode<Integer> node, int d)
2     if (node == null) {
3         return false;
4     }
5     if (node.hasLeft() && d == 0) {
6         return true;
7     }
8     if (node.hasRight() && d == 0) {
9         return true;
10    }
11    return isDeeper(node.getLeft(), d-1) ||
12           isDeeper(node.getRight(), d-1);
13 }
14
15 print(isDeeper(tree, 2)); // true
16 print(isDeeper(tree, 3)); // false

```

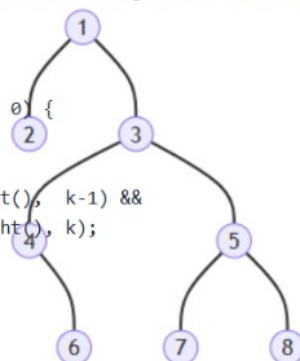


"עץ K-שמאלי" הוא עץ בינרי ובו כל מסלול משרשר העץ לעלה כלשהו מכיל לכל היותר K קשתות הפונות שמאלה (כלומר מקסימום K בנים שמאליים).
א. סרטטו "עץ 1-שמאלי" ובו 8 צמתים ומקסימום 4 רמות.
ב. כתבו פעולה חיצונית המחזירה אמת אם העץ הוא "עץ K-שמאלי".

```

1 public static boolean isLeftK(BinNode<Integer> node, int k)
2     if (node == null) {
3         return true;
4     }
5     if (node.hasLeft() && k == 0) {
6         return false;
7     }
8     return isLeftK(node.getLeft(), k-1) &&
9            isLeftK(node.getRight(), k);
10 }

```



האם השפות רגולריות?

$L_1 = \{ a^n b^m a^k \mid n, m, k > 0, n = k \}$ לא רגולרית
 $L_2 = \{ a^n b^m a^k \mid n, m, k > 0, n \% 2 = k \% 2 \}$ כן רגולרית
 $L_3 = \{ a^n b^m a^k \mid n, m, k > 0, n+m+k < 100 \}$ כן רגולרית

יש כמות סופית של קומבינציות (כמות האותיות חסומה ולכן סופית)
אפשר לבנות אס"ד לשפה
יש תלות, אך "תלות סופית"
כמות השאריות סופית ואפשר לבנות אס"ד

יש תלות "אין-סופית" בכמויות
לכן חייבים מחסנית במכונה

האם השפה $L_1 \cap L_3$ רגולרית? כן

האם השפה $L_1 \cap \overline{L_3}$ רגולרית? לא

האם השפה $L_2 \cap \overline{L_3}$ רגולרית? כן

כמות המילים היא אין סופית
ובכל מילה של חיתוך כמות ה-a-ים
בהתחלה זהה לכמות ה-a-ים בסוף
שזה מחייב מחסנית

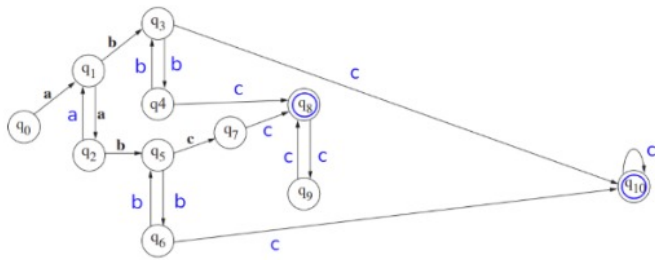
זה חיתוך, לכן כמות המילים
לא גדולה מזה שבשפה L_3
ו- L_3 שפה עם כמות מילים סופית
לכן רגולרית

$L = \{ a^m b^k c^x \mid m, k, x > 0 \text{ וגם } (m \% 2 = k \% 2 \text{ או } m \% 2 = x \% 2) \}$

abbac כמות ה-a-ים אי-זוגית לא בשפה
 וכמות ה-b-ים וגם ה-c-ים זוגית
 abc כן בשפה
 bbc כמות ה-a-ים אי-זוגית לא בשפה
 וגם כמות ה-b-ים אי-זוגית
 abcc כן בשפה
 לא תואם את התבנית
 של השפה, אחרי b
 חייב להיות c
 abbcc לא בשפה

מילה חייבת להתחיל
באות a. כי $m > 0$

$$L = \{a^m b^k c^x \mid m, k, x > 0 \text{ וגם } (m\%2 = k\%2 \text{ או } m\%2 = x\%2)\}$$



נתונות שלוש שפות L_1, L_2, L_3 מעל הא"ב $\{a, b, c\}$

עבור כל הטענה קבעו
אם היא נכונה או לא נכונה

$$R(R(L_1) \cdot R(L_2)) = L_1 \cdot L_2 \quad \text{לא נכון}$$

$$122333 \cdot 111223 \neq 111223 \cdot 122333$$

$$L_1 \cdot (L_2 \cup L_3) = (L_1 \cdot L_2) \cup (L_1 \cdot L_3) \quad \text{כן נכון}$$

מאוד דומה לפילוג מעל איחוד

$$L_1 \cap (L_2 \cdot L_3) = (L_1 \cap L_2) \cdot (L_1 \cap L_3) \quad \text{לא נכון}$$

$$L_1 = \{a, bc\}$$

$$L_2 = \{b\}$$

$$L_3 = \{c\}$$

$$L_1 \cap (L_2 \cdot L_3) = \{bc\}$$

$$(L_1 \cap L_2) \cdot (L_1 \cap L_3) = \emptyset$$

$$L = \{a^i b^j c^i - j \mid i \geq 4, (i - j) \geq 3\}$$

